Full Stack Record Content

S.No	Date	Program Title
1	28/11/2025	Personal Portfolio Website using HTML and CSS
2	03/12/2025	Smart Shopping Cart Application using React.js Components
3	10/12/2025	Internship Application Form by Implementing Event Handling and Form Validation in React.js
4	15/12/2025	Student Dashboard Using React Router with Multi-page Applications
5	22/12/2025	Smart Task Tracker Application Using React and Redux
6	26/12/2025	Online Job Application Form Using Angular
7	05/01/2026	Hospital Management System for Patients using AngularJS Services and REST API
8	12/01/2026	Simple Attendance Tracking System using Node.js: Installation and HTTP Server Setup
9	20/01/2026	Hospital Appointment RESTful API using Express.js: routing, middleware and Error Handling
10	23/01/2026	CRUD operations with MongoDB for Students Registration

11	28/01/2026	Online Examination Portal: Authentication and Session Management Using JWT in Node.js Applications
12	04/02/2026	Placement Registration Website Using React.js Frontend with Node.js Backend APIs
13	09/02/2006	Deployment of a Full-Stack Student Admission Application on a Cloud Platform
14	12/02/2026	Performance Profiling and Debugging of a Single Page Application Using React.js
15	20/02/2026	React To-Do App with Git Version Control
16	28/02/2026	Build and Deploy a Full Stack Web Application – Online Shopping

1. **Personal Portfolio Website using HTML and CSS**

Aim:

To design and implement a simple personal portfolio website using HTML and CSS with a structured layout including header, navigation menu, content area, and footer for showcasing resume details.

Algorithm:

Step 1: Start the process.

Step 2: Create a project folder and open it in an editor.

Step 3: Create index.html with `<html>`, `<head>`, and `<frameset>` tags for dividing the page into header, menu, content, and footer.

Step 4: In header.html, use `<h1>` or `<font>` tags to display the portfolio name and title.

Step 5: In menu.html, use `<a>` tags with `target='content'` for navigation links (Objective, Education, Projects, etc.).

Step 6: In content.html, insert a welcome message and an image using the `<img>` tag.

Step 7: Create individual section files (objective.html, education.html, projects.html, etc.) and structure them using `<h2>`, `<p>`, `<ul>`, `<li>` tags.

Step 8: In footer.html, add the copyright line using `<footer>` or `<center>` tag.

Step 9: Save all files with `.html` extension.

Step 10: Execute index.html in a web browser to verify frames and navigation.

Step 11: End.

Result:

Thus the above personal portfolio website using HTML and CSS is executed Successfully. Verified by the displayed output.

2. Smart Shopping Cart Application using React.js Components

Aim:

To design and implement a Smart Shopping Cart application using React.js functional components, demonstrating the use of JSX, props, and state to display products, add items to a cart, remove items, and calculate the total price dynamically.

Algorithm:

Step 1: Start the process.

Step 2: Create a React project and open it in a code editor.

Step 3: Create index.jsx and render the main App component using ReactDOM.createRoot().

Step 4: Create App.jsx as a functional component and initialize state using the useState hook to manage cart items.

Step 5: Define functions such as addToCart() and removeFromCart() to update the cart state.

Step 6: Create ProductCard.jsx as a functional component and pass product details and functions from App.jsx using props.

Step 7: Create Cart.jsx as a functional component and receive cart data and remove function through props.

Step 8: Use props to display product and cart information across components.

Step 9: Use state to dynamically update the user interface when products are added or removed.

Step 10: Apply styling using CSS to enhance the layout and appearance of the application.

Step 11: Execute the React application and verify the working of state, props, and functions.

Step 12: End.

Result:

Thus the above Developing Smart Shopping Cart using React.js components: functional components, props, and state is executed Successfully. Verified by the displayed output.

3. Internship Application Form by Implementing Event Handling and Form Validation in React.js

Aim:

To develop an Internship Application Form using React.js with event handling and form validation.

Algorithm:

Step 1 : Create a React application and add the InternshipApplication component inside App.jsx.

Step 2 : Initialize state variables (formData, errors, submitted) using useState.

Step 3 : Design the form with input fields, select dropdown, file upload, and checkbox.

Step 4 : Create a handleChange function to update formData based on user input.

Step 5 : Handle different input types such as text, number, checkbox, and file inside handleChange.

Step 6 : Create a validate function to check all form fields.

Step 7 : Add validation rules for name, email, mobile, college, CGPA, domain, resume, and agreement.

Step 8 : Create a handleSubmit function and prevent default form submission using preventDefault().

Step 9 : Call the validate function and store error messages in the errors state.

Step 10 : If there are no errors, display a success message indicating successful submission.

Result:

Thus the above Internship application form by Implementing event handling and form validation in React.js is executed Successfully. Verified by the displayed output.

4. Student Dashboard Using React Router with Multi-page Applications

Aim:

To design and implement a Student Dashboard using React that functions as a multi-page Single Page Application (SPA), providing navigation between Dashboard, Students, and Profile sections without reloading the webpage.

Algorithm:

Step 1: Start the process.

Step 2: Create a React project and open it in a code editor.

Step 3: Create a functional component named StudentDashboard.

Step 4: Initialize a state variable using useState() to store the current page.

Step 5: Design the sidebar with navigation buttons (Dashboard, Students, Profile).

Step 6: Add on Click events to update the state when a button is clicked.

Step 7: Check the state value and display the corresponding content using conditional rendering.

Step 8: Display student details and profile information in the content area.

Step 9: Apply CSS styling for layout and user interface.

Step 10: Run the application and verify page navigation without reload.

Step 11: End the process.

Result:

Thus the above Student Dashboard using react router with multi-page applications is executed successfully. Verified by the displayed output.

5. Smart Task Tracker Application Using React and Redux

Aim:

To design and develop a Task Management Application using React.js and Redux that allows users to add, delete, and update tasks efficiently using centralized state management.

Algorithm:

Step 1 : Start the process

Step 2 : Initialize the Redux store.

Step 3 : Define action types such as ADD_TASK, DELETE_TASK, TOGGLE_TASK

Step 4 : Create action creators to dispatch task-related actions

Step 5: Create a reducer function to handle Adding a new task , Deleting an existing task ,Toggling task completion status

Step 6 : Combine reducer with middleware and create Redux store.

Step 7 : Wrap the root component (App) with Redux Provider.

Step 8 : Create input field to enter task details.

Step 9 : When user clicks "Add":Dispatch ADD_TASK action,Store updates the state

Step 10: Display all tasks using useSelector.

Step 11: When user clicks on task:Dispatch TOGGLE_TASK action,Update task completion status

Step 12 : When user clicks "Delete":Dispatch DELETE_TASK action,Remove task from state

Step 13 : Render updated task list automatically.

Step 14 : Stop the application.

Result:

Thus the above “Smart Task Tracker Application Using React and Redux” is executed Successfully Verified by the displayed output.

6. Online Job Application Form Using Angular

Aim:

To design and develop an Online Job Application Form using Angular that collects applicant details, validates user input, and ensures all mandatory information is completed before submission.

Algorithm:

Step 1: Start the process.

Step 2: Create a new Angular project and open it in a code editor.

Step 3: Design the form layout using and <form> and display the title.

Step 4: Add personal detail fields using <input> and <textarea> to collect name, email, phone number, and address.

Step 5: Apply validation using required, pattern, and [(ngModel)] to ensure correct email format and a 10-digit phone number.

Step 6: Provide job details using <select> dropdowns for position and preferred location.

Step 7: Add educational qualification fields using <select> elements.

Step 8: Display skills using <input type="checkbox"> and generate options dynamically using *ngFor.

Step 9: Provide experience selection using <input type="radio"> and display additional fields using *ngIf.

Step 10: Include resume upload using <input type="file"> and add a declaration checkbox.

Step 11: Submit the form and enable the submit button only when the form is valid.

Step 12: Display a confirmation message after successful submission.

Step 13: End the process.

Result:

Thus the above “Online Job Application Form Using Angular” is executed Successfully. Verified by the displayed output.

7. Hospital Management System for Patients using AngularJS Services and REST API

Aim:

To design and implement a Hospital Management System for Patients using AngularJS for the frontend and Node.js with Express.js for the backend, by applying RESTful API principles to manage patient records, enabling users to add and view patient details efficiently while maintaining smooth and reliable communication between the client and server.

Algorithm:

Step 1: Start the process.

Step 2: Create a project folder and initialize a Node.js project using npm init.

Step 3: Install Express.js using npm install express and create server.js as the main backend file.

Step 4: Import Express.js and create an application instance using express().

Step 5: Configure built-in middleware express.json() to parse incoming JSON request bodies.

Step 6: Create an empty array patients to store patient records and a variable id to assign unique patient IDs.

Step 7: Implement a GET route /patients to fetch and return all patient records in JSON format.

Step 8: Implement a POST route /patients to accept patient details (Name, Age, Gender), assign a unique ID, and store the record in the patients array.

Step 9: Create index.html for the frontend interface, including input fields for patient Name, Age, and Gender, a button to add patients, and a table to display patient records.

Step 10: Include AngularJS in index.html and define a module (app) and controller (ctrl) to manage frontend logic.

Step 11: Implement $scope.add() function to send new patient data to the backend using $http.post("/patients").

Step 12: Implement load() function to fetch all patient records from the backend using $http.get("/patients") and bind the data to display in the table.

Step 13: Clear input fields after adding a patient and reload the patient list.

Step 14: Start the server using node server.js and open http://localhost:3000 in a web browser.

Step 15: Test the system by adding patient details and verifying that records appear correctly in the table.

Step 16: End the process.

Result:

Thus the above "Hospital Management System for patients using AngularJS Services and REST API" program is executed Successfully. Verified by the displayed output.

8. Simple Attendance Tracking System using Node.js: Installation and HTTP Server Setup

Aim:

To create a Simple Attendance Tracking System using Node.js: Installation and HTTP Server Setup that responds with a message in the web browser.

Algorithm:

Step1: Start the process.

Step2:Install Node.js in the system.

Step3: Open Command Prompt.

Step4: Create a new React application using npx create-react-app.

Step5: Navigate to the project folder using cd command.

Step6: Open the project in a code editor (VS Code).

Step7: Locate the App.js file inside the src folder.

Step8: Remove the existing code inside App.js.

Step9:Write a functional component that returns JSX content.

Step10: Export the App component using export default App.

Step11: Run the application using npm start.

Step12: Open the browser at http://localhost:3000 to view the output.

Result:

Thus, Node.js was successfully installed and a simple HTTP server was created and executed successfully.

9. Hospital Appointment RESTful API using Express.js: routing, middleware and Error Handling

Aim:

To design and implement a Hospital Appointment RESTful API using Express.js by applying routing to manage API endpoints, middleware to process incoming requests, and error handling to ensure reliable and secure communication between the client and server.

Algorithm:

Step 1: Start the process.

Step 2: Create a project folder and initialize a Node.js project using npm init.

Step 3: Install Express.js and create index.js as the main server file.

Step 4: Import Express.js and create an application instance using express().

Step 5: Configure built-in middleware express.json() to parse JSON request bodies.

Step 6: Create a custom middleware (logger.js) to log the HTTP method and request URL for every incoming request.

Step 7: Create a routing file appointmentRoutes.js and define API endpoints using Express Router.

Step 8: Implement routing to handle viewing doctors, booking appointments, and cancelling appointments using GET, POST, and DELETE methods.

Step 9: Add input validation and return appropriate responses to handle invalid requests as part of error handling.

Step 10: Implement a global error-handling middleware in index.js to handle runtime errors.

Step 11: Start the server using node index.js and listen on the specified port.

Step 12: Test the API endpoints using a browser or API testing tools such as Postman.

Step 13: End.

Result:

Thus the above Developing a Hospital Appointment RESTful API using Express.js: routing, middleware and error handling is executed Successfully. Verified by the displayed output.

10. CRUD operations with MongoDB for Students Registration

Aim:

To develop a full-stack authentication system using React.js (frontend) and MongoDB Atlas (database) that allows users to register, login, and access a protected home page using JWT authentication.

Algorithm:

Step 1: Start the program.

Step 2: Start the Node.js server and configure Express middleware (CORS and JSON parser).

Step 3: Establish connection with MongoDB Atlas using Mongoose.

Step 4: Create a User schema containing name, email, and password fields.

Step 5: When a user registers, receive name, email, and password from the frontend.

Step 6: Check if the email already exists in the database.

Step 7: Hash the password using bcrypt and store the new user in MongoDB.

Step 8: When a user logs in, receive email and password from the frontend.

Step 9: Validate user credentials by comparing the entered password with the hashed password.

Step 10: If credentials are valid, generate a JWT token and send it to the frontend.

Step 11: Store the received token in localStorage on the client side.

Step 12: When accessing the Home page, verify the token by sending it to the backend.

Step 13: If the token is valid, allow access; otherwise, remove the token and redirect to the login page.

Step 14: Stop the process

Result:

Thus the above Database CRUD operations with MongoDB/MYSQL via Node.js is executed Successfully. Verified by the displayed output.

11. Online Examination Portal: Authentication and Session Management Using JWT in Node.js Applications

Aim:

To develop a secure Online Examination Portal: Authentication and Session Management Using JWT in Node.js Applications

Algorithm:

1. Start the process.
2. Create a new project directory for the Online Exam Portal.
3. Initialize the Node.js application inside the directory using npm init.
4. Install required packages such as Express, JWT, bcrypt, CORS, and Nodemon.
5. Create the server file (server.js) and configure the Express server.
6. Import required modules and initialize middleware.
7. Accept user registration details (username and password).
8. Hash the password using bcrypt and store user data.
9. Verify login credentials and generate a JWT token upon successful authentication.
10. Store the generated JWT token in the client-side local storage.
11. Verify the token when accessing the exam route and display questions if authentication is successful.
12. End

Result:

Thus the above Online Examination Portal: Authentication and Session Management Using JWT in Node.js Applications is executed Successfully. Verified by the displayed output.

12. Placement Registration Website Using React.js Frontend with Node.js Backend APIs

Aim:

To develop a Placement Registration Website using React as frontend and Node.js with Express as backend, where student details are collected through a form and stored on the server.

Algorithm:

Step 1: Start the program.

Step 2: Create a React frontend application to design the registration form.

Step 3: Create input fields such as Name, Email, and Department.

Step 4: When the user fills the form and clicks the Register button, collect the form data using React state.

Step 5: Send the collected data to the backend server using Axios HTTP POST request.

Step 6: Create a Node.js backend server using Express.

Step 7: Enable CORS and JSON middleware to handle incoming data.

Step 8: Receive the student data from the frontend.

Step 9: Store the received data in a JSON file on the server using the File System module.

Step 10: Send a success response back to the frontend.

Step 11: Display a success message to the user.

Step 12: Stop the program.

Result:

Thus the above Integrating React.js/AngularJS front-end with Node.js backend APIs is executed Successfully. Verified by the displayed output.

13. Deployment of a Full-Stack Student Admission Application on a Cloud Platform

Aim:

To develop and deploy a **Deployment of a Full-Stack Student Admission Application on a Cloud Platform** using React.js as the frontend, Node.js with Express.js as the backend, and SQLite as the database, where student details are collected, stored, retrieved, and managed through RESTful APIs hosted on a cloud platform.

Algorithm:

Step 1: Start the program.

Step 2: Create a React frontend application to design the Student Admission interface.

Step 3: Create input fields such as Student Name and Phone Number.

Step 4: Use React useState() to store input values and student records.

Step 5: When the user clicks the Add Student button, collect the form data from React state.

Step 6: Send the collected data to the backend using Axios POST request (/add-user).

Step 7: Create a Node.js backend server using Express framework.

Step 8: Enable CORS and JSON middleware to handle cross-origin and JSON data requests.

Step 9: Connect to the SQLite database and create the students table if it does not exist.

Step 10: Receive and validate student data in the backend, then insert it into the database.

Step 11: Send a success response back to the frontend after successful insertion.

Step 12: Use a GET API (/users) to retrieve all student records and display them in the frontend.

Step 13: Implement DELETE API (/delete-user/:id) to remove selected student records from the database.

Step 14: Deploy the backend and frontend to a cloud platform and access the application through the public URL, then stop the program.

Result:

Thus, the Full-Stack Student Admission Application was successfully developed and deployed on a cloud platform with verified integration between the React frontend and Node.js backend through successful add, view, and delete operations.

14. Performance Profiling and Debugging of a Single Page Application Using React.js

Aim:

To design and implement a React.js web application to demonstrate debugging and performance profiling techniques by identifying unnecessary component re-rendering and optimizing performance using console.log() and React.memo().

Algorithm:

Step 1: Start the process.

Step 2: Create a new React application using create-react-app.

Step 3: Open the project in a code editor.

Step 4: Create index.jsx and render the main App component using ReactDOM.createRoot().

Step 5: Create App.jsx as a functional component.

Step 6: Initialize state variables using the useState hook (for example: count and text).

Step 7: Create a Counter component and pass state and update functions using props.

Step 8: Create an ExpensiveComponent that performs a heavy calculation using a large loop.

Step 9: Add console.log() statements inside components to observe re-rendering behavior.

Step 10: Run the application and open the browser console to monitor component rendering logs.

Step 11: Observe that the ExpensiveComponent re-renders unnecessarily when typing in the input field.

Step 12: Optimize performance by wrapping ExpensiveComponent with React.memo() to prevent unnecessary re-rendering.

Step 13: Re-run the application and verify improved performance in the browser console.

Step 14: Confirm that the heavy component renders only when required.

Step 15: End the process.

Result:

Thus, the Performance Optimization React Application was successfully implemented.

Unnecessary component re-rendering was identified using console debugging and optimized using React.memo(), resulting in improved application performance

15. React To-Do App with Git Version Control

Aim:

To develop a simple React-based To-Do application that allows users to add, view, and delete tasks while using Git for version control and collaborative workflow.

Algorithm :

Step 1: Start a new React project.

Step 2: Open it in VS Code.

Step 3: Initialize Git in the project.

Step 4: Connect Git to GitHub.

Step 5: Create an input box to type tasks.

Step 6: Add a button called “Add” to submit tasks.

Step 7: Show the list of tasks below the input.

Step 8: Make typing in input save the task**.**

Step 9: Make clicking “Add” put the task in the list.

Step 10: Add a “Delete” button next to each task to remove it.

Step 11: Test everything and push changes to GitHub.

Result:

Thus the above React to do app with Git Version is executed Successfully. Verified by the displayed output.

16. Build and Deploy a Full Stack Web Application – Online Shopping

Aim:

To design, develop, and deploy a Full Stack Online Shopping Web Application using React.js for the frontend, Node.js and Express.js for the backend, and MongoDB as the database, and to host the application on a cloud platform for public access.

Algorithm:

Step 1: Start the process.

Step 2: Set up the development environment by installing Node.js, npm, and MongoDB.

Step 3: Create a project folder and divide it into two directories: frontend and backend.

🔹 **Backend Development**

Step 4: Initialize a Node.js project inside the backend folder using npm init -y.

Step 5: Install required dependencies such as express, mongoose, and cors.

Step 6: Create a server.js file to configure the Express server.

Step 7: Connect the backend application to MongoDB using Mongoose.

Step 8: Define a Product schema and model to store product details (name, price, image, etc.).

Step 9: Create REST API routes such as:**

- GET – to fetch products
- POST – to add products

Step 10: Run the backend server and verify API functionality using browser or Postman.

🔹 **Frontend Development**

Step 11: Create a React application using create-react-app inside the frontend folder.

Step 12: Install required packages such as axios for API communication.

Step 13: Create functional components such as App.jsx, ProductList.jsx, and Cart.jsx.

Step 14: Use useState hook to manage cart and product states.

Step 15: Fetch product data from backend APIs using Axios.

Step 16: Display products dynamically using JSX and props.

Step 17: Implement Add to Cart and Remove from Cart functionality.

Step 18: Calculate total price dynamically using JavaScript array methods.

Step 19: Apply CSS styling to enhance user interface design.

Step 20: Run the React application and verify frontend-backend integration.

- **Deployment Process**

Step 21: Push the backend and frontend code to a GitHub repository.

Step 22: Deploy the backend to a cloud platform (such as Render/Railway).

Step 23: Configure environment variables (e.g., MongoDB connection string).

Step 24: Deploy the frontend to a hosting platform (such as Netlify/Vercel).

Step 25: Update API base URL in frontend to point to deployed backend.

Step 26: Test the deployed application in a browser to ensure full functionality.

Step 27: End the process.

Result:

Thus the above program full stack web application- an online shopping is executed Successfully. Verified by the displayed output.